



Samplers

Сэмплеры



- Сэмплеры представляют собой специальные формы, которые используются для идентификации текстуры при выполнении текстурирования. Значение сэмплера указывает на номер текстурного модуля, к которому осуществляется доступ.
- Тип сэмплера определяет тип цели (target) для текстурного модуля. Текстура, привязанная к указанной цели данного текстурного модуля, будет использоваться при выборке.
- Привязка текстурных объектов к целям выполняется стандартной функцией `BindTexture`. Выбор текстурного модуля для привязки осуществляется с помощью `ActiveTexture`. Локацию сэмплера можно получить с помощью `GetUniformLocation`. Значения сэмплеров должны устанавливаться вызовом `Uniform1i{v}`.
- Запрещено иметь переменные разных типов сэмплеров, указывающие на один и тот же текстурный модуль в пределах одного программного объекта.
- Активные сэмплеры - это сэмплеры, фактически используемые в программном объекте. Команда `LinkProgram` определяет, является ли сэмплер активным. `LinkProgram` также проверяет, не превышает ли количество активных сэмплеров в шейдере(ах), содержащихся в программном объекте, максимально допустимые пределы. Если количество активных сэмплеров превышает допустимые пределы, компоновка завершается неудачно.

Изображения

- Изображения представляют собой специальные формы, которые идентифицируют определенный уровень текстуры для чтения или записи с использованием встроенных функций `imageLoad`, `imageStore` или атомарных операций. Значение `image`-переменной представляет собой целое число, указывающее на номер используемого модуля изображения (`image unit`). Нумерация модулей изображения начинается с нуля, а их максимальное количество зависит от реализации (определяется значением `MAX_IMAGE_UNITS`). Важно отметить, что модули изображений для `image`-переменных независимы от текстурных модулей, используемых для сэмплов - их количество в реализации может различаться. Текстуры привязываются к модулям изображений и текстурным модулям отдельно и независимо. Тип переменной изображения должен соответствовать целевой текстуре изображения, привязанного в данный момент к объекту изображения. Иначе результат операций `load`, `store` или атомарных операций будет неопределенным.
- Локацию `image`-переменной необходимо запрашивать с помощью `GetUniformLocation`. Значения `image`-переменных устанавливаются вызовом `Uniform1i{v}`.
- В отличие от сэмплов, нет ограничений на количество активных `image`-переменных, которые могут использоваться программой или отдельным шейдером.

Доступ к памяти в шейдерах



Шейдеры могут выполнять произвольный доступ к памяти буферных объектов следующими способами:

1. Через шейдерные `buffer`-переменные:
 - Чтение значений
 - Запись значений
 - Атомарные операции
2. Через встроенные функции работы с `image`-переменными:
 - `imageLoad` - чтение из текстур/буферов
 - `imageStore` - запись в текстуры/буферы
 - Атомарные операции с изображениями
3. Через атомарные операции с атомарными счетчиками переменных.

Возможность такого произвольного доступа к памяти в высокопараллельных графических конвейерах создает проблемы с:

- Порядком выполнения операций
- Синхронизацией между шейдерными вызовами

Особенности реализации:

- Доступ может осуществляться как к текстурам, так и к буферным объектам
- Поддерживаются как обычные операции, так и атомарные
- Разные типы шейдеров имеют различные ограничения на доступ к памяти
- Требуется явная синхронизация при совместном доступе

Порядок доступа к памяти в шейдерах

Порядок чтения и записи в текстуры или буферные объекты шейдерами в основном не определён. Для некоторых типов шейдеров даже количество вызовов шейдера, выполняющих операции с памятью, может быть неопределённым. Конкретные правила:

1. **Вершинные и тесселяционные шейдеры:**
 - Гарантированно выполняются минимум один раз для каждой уникальной вершины. Могут выполняться несколько раз по причинам, зависящим от реализации. При повторном использовании вершины шейдер может выполняться только один раз.
2. **Фрагментные шейдеры:**
 - Количество вызовов зависит от нескольких факторов. Не выполняются, если фрагмент:
 - Не проходит тест владения пикселем
 - Отбрасывается scissor-тестом
 - Отбрасывается операциями мультисэмплинга
 - При включенных ранних тестах фрагментов шейдер не выполняется для отброшенных фрагментов. Без мультисэмплинга - ровно один вызов на фрагмент. С мультисэмплингом - от 1 до N вызовов (N - количество сэмплов фрагмента). При определении затенения для каждой выборки вызывается ровно N раз.
3. **Специальные случаи:**
 - Вызовы для вычисления производных не влияют на операции записи, атомарные операции и обновления счётчиков.
4. **Порядок выполнения:**
 - Относительный порядок вызовов одного типа шейдеров не определён. Запись для примитива B может завершиться раньше записи для примитива A, даже если A был задан раньше B. Для фрагментных шейдеров: выводы записываются в буфер кадра в порядке примитивов, но операции store - нет. Порядок между разными типами шейдеров в основном не определён. Исключение: когда входы шейдера генерируются предыдущей программируемой стадией, гарантируется завершение соответствующих вызовов предыдущего шейдера. Для geometry shader гарантируется только генерация вершин для текущего обрабатываемого фрагментным шейдером примитива.

Ограничения порядка выполнения и синхронизация шейдеров

- Описанные ограничения порядка выполнения шейдерных вызовов делают невозможными некоторые формы синхронизации между вызовов в пределах одного набора примитивов.
- Операции записи в разные области памяти в пределах одного шейдерного вызова могут становиться видимыми другим вызовам не в порядке их выполнения. Для решения этой проблемы существует встроенная функция `memoryBarrier`, которая обеспечивает строгий порядок операций чтения/записи в пределах одного вызова. Вызов `memoryBarrier` гарантирует, что:

Все операции с памятью, инициированные до барьера, завершатся

Только после этого начнут выполняться операции, следующие после барьера.

Типы барьеров:

- `SHADER_STORAGE_BARRIER_BIT` - Гарантирует видимость записей в SSBO.
- `TEXTURE_FETCH_BARRIER_BIT` - Синхронизирует доступ к текстурам.
- `FRAMEBUFFER_BARRIER_BIT` - Обеспечивает порядок операций с FBO.
- `ATOMIC_COUNTER_BARRIER_BIT` - Синхронизирует атомарные счетчики.
- `ALL_BARRIER_BITS` - Полная синхронизация для всех операций.

Оптимизация: `MemoryBarrierByRegion()`. Работает только для фрагментных шейдеров и ограничивает зону синхронизации.

Реализация кэширования и согласованности памяти

Особенности кэширования в реализации

Графические процессоры могут использовать несколько отдельных кэшей для:

- Текстурных данных
- Вершинных атрибутов
- Шейдерных операций с памятью

Эти кэши **не обязаны** автоматически поддерживать согласованность с записями из шейдеров. Это означает:

1. Записи одного вызова могут временно оставаться в локальном кэше процессора
2. Старые значения могут сохраняться в других кэшах системы
3. Видимость изменений для других стадий конвейера не гарантируется

Когерентность памяти

Для общих данных между вызовами шейдеров:

- Используйте **coherent** в GLSL.
- Вызывайте **memoryBarrier()** в шейдере после записи.

Что использовать:

Ситуация

Шейдер пишет → другой шейдер читает:

Передача данных между рендер-пассажами:

Фрагментные шейдеры:

Решение

coherent + MemoryBarrier

MemoryBarrier с нужным битом

MemoryBarrierByRegion